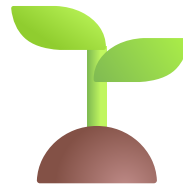


Let's get introduced.

Chanakya University · Data Mining & Data Warehousing · Module 4



If you had the world's data — what would you build?

Turn to the person next to you. 60 seconds. Anything goes.



BEFORE WE BEGIN

Where we are. Where we're going.

A quick bridge from what you already know to what you're about to build.

MODULE 1 — WHAT YOU COVERED

- 1 **Data Warehousing fundamentals** — ETL pipelines, star schema, OLAP cubes, fact & dimension tables
 - 2 **Classical Data Mining** — classification, clustering, association rules on clean structured data
 - 3 **The limitations** — SQL equality breaks on typos, batch ETL breaks on real-time streams, no tool for unstructured text
- ?
- The open question Module 1 left unanswered: *what do you do when the data is messy, massive, and unstructured?*

MODULE 4 — TODAY'S ANSWERS

Text Mining

Clean 5M dirty handles with MinHash & LSH

Graph Analytics

Rank a network without being fooled by bots

Hybrid AI RAG

Answer questions with zero hallucinations

THE THREAD

One manufacturing plant. Two failures. The rest of this deck is the engineering fix.

MODULE 4 · DATA MINING & DATA WAREHOUSING

Web Mining, Graph Analytics & AI Integration

Real-World Engineering Case Study: Netflix Live Sports Data Platform Engine (5 Million Subscribers).

Netflix Live Sports

HF LLM Course

MinHash LSH

PageRank

DuckDB RAG

One Lie, Caught Manually. Now Multiply by 847.

A floor supervisor uncovers a false report in 40 minutes. The real question: what happens when there are 847 employees?

1 END OF SHIFT

Ravi submits his daily log — **200 quality checks**, all units passed, zero defects. Looks clean.

2 SUPERVISOR CROSS-CHECKS

All 200 timestamps fall in a **4-minute window**. Sensor log: conveyor idle for 3 hours. CCTV: Ravi left the floor at 2 PM.

3 CASE CLOSED — FOR ONE PERSON

40 minutes of manual work. Three sources reconciled. Report confirmed fabricated. Problem solved.

 THE REAL QUESTION

847 employees. 3 shifts. 50+ data points each. That is **42,350 records per day**. Manual audit does not scale.

23

working days to audit one day's records

THE MATH

$40 \text{ min} \times 847 = 565 \text{ hrs}$

$565 \div 24 \text{ hr/day} = \mathbf{23}$

days

DATA MINING

Automates this — pattern detection at scale, in seconds.

Same Factory. Same Quarter. Four Different Answers.

Data Mining caught the lie. But two hours later, the Director asked one simple question — and the entire plant had no answer.

2 HOURS AFTER CATCHING RAVI

The Factory Director walks in. One question: "**What was our actual defect rate in Q3?**"

FOUR TEAMS. FOUR SYSTEMS. FOUR ANSWERS.

2.3% Excel — Floor Sup.	3.8% QMS Software
1.9% ERP System	4.1% HR — Rework Hours

WHY ARE THEY ALL DIFFERENT?

Each team used different time ranges, different defect definitions, and different update schedules. Same factory. Same quarter. **Four realities.**

THE FIX

One ETL pipeline pulls from all 4 systems nightly, applies agreed definitions, and loads into a single **Data Warehouse** — so the Director gets one number, every time.

THE DISTINCTION

Data Mining

Finds the lie in today's data.

Data Warehousing

Makes sure last quarter tells the same story as today.

Ravi's Factory Had This Problem. So Did NASA and Wall Street.

The same two failures — no anomaly detection, no single source of truth — have cost the world hundreds of millions of dollars.

KNIGHT CAPITAL — \$440M LOSS, 2012

No anomaly detection at scale

An old, uncleaned trading flag was accidentally reactivated. No automated system caught it. Within 45 minutes, 4 million inadvertent stock trades were executed — the firm went bankrupt. **That is the Data Mining problem.**

MARS CLIMATE ORBITER — \$125M LOSS, 1999

No single source of truth

NASA's navigation software used Newtons. The contractor's software used pound-force. No warehouse enforced agreed units. The spacecraft burned up on entry. **That is the Data Warehousing problem.**

WHAT THIS MEANS FOR THE FACTORY

Ravi and the Director are not edge cases

Every organisation has a Ravi — someone whose anomaly goes undetected at scale. Every boardroom has a Director — someone who gets four different answers to one question. Data Mining and Data Warehousing are the engineering response to both.

Why DM & DW Exist: The 10 Fundamental Data Problems

Ten inescapable bottlenecks that forced computer scientists to invent data warehousing.

01. DATA SILOS

Isolated Storage

Sales, support & payments locked in separate DBs with no unified view.

02. VOLUME OVERFLOW

Memory Bounds

Terabytes of daily logs exceeding RAM capacity of standard servers.

03. VELOCITY SURGE

Streaming Friction

Real-time event streams crashing traditional overnight batch ETL jobs.

04. UNSTRUCTURED FLOOD

80% Non-SQL Data

Twitch chats, PDFs & social feeds that cannot fit into neat SQL columns.

05. IDENTITY CRISIS

Noisy Duplication

Same customer registering under 10 handles
(`lew_hamilton_44` vs `L3wis`).

06. DATA CORRUPTION

Noise & Typos

Missing values, leet-speak, and corrupted log timestamps.

07. HIGH DIMENSIONS

Curse of Dimensionality

Thousands of raw feature columns breaking traditional machine learning.

08. TOPOLOGY COMPLEXITY

Graph Networks

Interconnected follower networks that SQL `GROUP BY` cannot rank.

09. AI HALLUCINATION

No Ground Truth

LLMs guessing made-up names when ungrounded by a warehouse.

Is Data Warehousing Still Relevant in the AI Era?

Why LLMs cannot replace Data Warehousing — they depend on it!

THE MYTH

"LLMs make Data Warehouses obsolete."

Some students think: *"We have ChatGPT and Gemini now, so why do we need SQL, MinHash, or Data Warehouses?"*

The reality: An LLM without a Data Warehouse hallucinates fake names, gets math wrong, and makes up dates. It is a smooth-talking witness who never checked the records.

THE FACT

Data Warehousing is more vital than ever.

A Data Warehouse is the **ground-truth fact backbone**. LLMs need warehouses to supply verified data context via Vector RAG.

THE MODERN PARADIGM

Data Warehousing cleans & structures the facts. LLMs synthesize & communicate the answers.

SCD Types: Where Classical DWH Hits a Wall

Slowly Changing Dimensions are the unglamorous heart of data warehousing. The new AI era handles them in ways that were impossible before.

THE CLASSIC PROBLEM — EMPLOYEE PRIYA MOVES FROM PUNE TO BENGALURU

- TYPE 1** **Overwrite** — update Priya's city to Bengaluru, delete Pune. Simple, but **history is gone forever**. The quarterly report will show she was always in Bengaluru. Wrong.
-
- TYPE 2** **Add a new row** — keep Pune row with end_date, insert new Bengaluru row. History preserved, but every ETL script now needs `WHERE is_current = 1` or it double-counts Priya in every join. **One missed WHERE clause crashes the entire quarterly report**.
-
- TYPE 3** **Add a new column** — keep both city_old and city_current. Only tracks one previous state. **Priya moves a third time — you are out of columns**.
-
- TYPE 6** **Hybrid** — combines Types 1+2+3. Maximum history, minimum data loss. Also **maximum maintenance cost**. A single schema change touches 6 ETL jobs, 4 stored procedures, and 2 reporting views.

WHERE LLM-ERA DWH WINS

Fuzzy entity resolution

"Priya Sharma, Pune" and "P. Sharma, Bengaluru" are matched as the same person using embeddings — no manual rule needed.

Semantic change detection

LLMs distinguish a *format change* (Bombay → Mumbai) from a *real move* (Pune → Bengaluru). Classical ETL cannot.

Natural language time-travel

"What was Priya's region in Q2 2023?" — answered via Text-to-SQL, no manual

Module 1 vs Module 4: What Was Left Unsolved?

What classical data warehousing solved, and why we needed the LLM paradigm shift.

CAPABILITY	MODULE 1 (CLASSICAL DM & DW)	MODULE 4 (LLM & HYBRID AI ERA)
Data Input	Clean SQL tabular rows only	Unstructured Twitch chats, JSON logs, PDFs & images
Deduplication	Exact SQL <code>WHERE a = b</code> (Fails on typos)	MinHash + LSH Banding (Sub-second O(N) fuzzy match)
Network Ranking	Naive SQL <code>COUNT(*)</code> (Tricked by bot farms)	PageRank Power Iteration & Louvain Modularity Cliques
Querying	Complex raw SQL strings	Natural Language Text-to-SQL + Vector Cosine Search
Fact Verification	Manual rule scripts	Dual-Storage DuckDB RAG (Zero Hallucination)

What We Will Cover in Today's Class

No dry recaps — we jump straight into building Netflix's live sports data platform.

PROBLEM 1 — TEXT MINING

Handle Deduplication

Cleaning 5M dirty handles (`lew_hamilton_44` vs `L3wis_Ham1lton`) using Character Bigrams, MinHash signatures, and $O(N)$ LSH Banding.

PROBLEM 2 — GRAPH ANALYTICS

Influencer Ranking

Finding the #1 top influencer for the live broadcast using PageRank power iteration ($d=0.85$) without getting tricked by 500k spam bots.

PROBLEM 3 — AI & VECTORS

Grounded Hybrid RAG

Answering complex questions ("Which paid subscribers discussed VIP passes?") using Hugging Face LLM transformers, pgvector, and DuckDB SIMD joins.

LEARNING OUTCOME

Complete Data Architecture

By solving these 3 problems step-by-step, you will master modern unstructured data mining and multi-cloud systems (AWS, Azure, GCP).

The Engineering Mission: Netflix Live Sports Engine

You are the Senior Data Architect at Netflix building the real-time data infrastructure.

- **Your Role:** Lead Data Architect at [Netflix Engineering](#).
- **The Platform:** Netflix is launching a real-time live global F1 sports streaming & community engine for 5 million fans.
- **The SLA:** Sub-second response time across 3 simple core business requirements.

THE THREE SIMPLE BUSINESS QUESTIONS

Q1 (Text): How do we clean millions of duplicate usernames so subscriber metrics aren't inflated?

Q2 (Graph): How do we find the #1 top influencer in the network to feature on the live broadcast (without being fooled by fake bot followers)?

Q3 (AI RAG): How do we answer subscriber queries like 'Which paid subscribers discussed VIP race passes?' with zero AI hallucinations?

Three Critical Engineering Bottlenecks

Naive relational database approaches failed completely at scale.

BOTTLENECK 1 **40 GB/day of raw JSON dumped into production PostgreSQL.** Transaction buffer locks starved connection pools. Dashboard times out every 20 mins.

BOTTLENECK 2 **Noisy handle duplication.** `lew_hamilton_44` , `Lewis_Hamilton_44` , and `hamilton_fan` register as separate users. SQL `LIKE` loop requires 25 trillion comparisons ($O(N^2)$).

BOTTLENECK 3 **Raw text uploaded directly to an LLM.** Asking "which paid subscribers discussed VIP paddock access?" caused the AI to hallucinate fake users and wrong paid amounts.

Data Stream Audit: Three Modalities

Inspecting the exact mathematical shapes of incoming target data streams.

STREAM A — UNSTRUCTURED TEXT

Twitch Scraper JSON & Chat Logs

40 GB / day JSON + unstructured text. Dirty handles, no schema. Must land in Data Lake (MinIO) first.

STREAM B — RELATIONAL LEDGER

Subscriber Payment Ledger

8 GB PostgreSQL. Clean `user_id`, `tier_paid_usd`, `days_active`. Perfect for exact SQL relational filters.

STREAM C — GRAPH NETWORK

Netflix Follow Network

5M nodes, 45M directed edges. SQL `GROUP BY` cannot compute recursive graph network rank.

Classroom Activity Overview

20-minute hands-on challenge to expose system traps before diving into theory.

TASK 01 — 5 MINS

Expose the System Traps

Explain why streaming raw JSON into OLTP crashes connection pools, why SQL LIKE wildcards fail at $O(N^2)$, and why direct LLM prompt engineering hallucinated.

TASK 02 — 10 MINS

Draw the Modern Data Architecture

Draw the 5-layer pipeline routing JSON, PostgreSQL ledger, and PDF text. Map each component to local Docker tools and all 3 cloud platforms (AWS, Azure, GCP).

TASK 03 — 5 MINS

Executive Memo

Draft a 2-sentence technical memo explaining why LLMs fail at numerical filtering and how your dual-storage DuckDB architecture guarantees zero hallucinations.

HINT CARD

Formula Reference

Jaccard $J(A,B) = |A \cap B| / |A \cup B|$. LSH probability $P(s) = 1 - (1 - s^r)^b$. PageRank $PR(u) = (1-d)/N + d \sum PR(v)/L(v)$. Cosine $\cos(A, B) = (||A|| ||B||)$

Required Reading & Foundational Papers

Read these authoritative guides and seminal research papers to deepen subject mastery.

TECHNICAL BLOG & COURSE DEEP-DIVES

[Hugging Face LLM Course Chapter 1: Introduction to LLMs](#)

Covers Encoder vs Decoder architectures, tokenizers, and self-attention mechanism.

[Arpit Bhayani: MinHash - Fast Jaccard Similarity at Scale](#)

SEMINAL COMPUTER SCIENCE PAPERS

- **LLM Agents:** Sharma et al. (2026) *"The State of LLM in Solving Data Mining Tasks"*
- **MinHash:** Broder et al. (1997) *"Syntactic Clustering of the Web"*
- **PageRank:** Page, Brin, Motwani, Winograd (1999) *"The PageRank Citation Ranking"*
- **Vector RAG:** Lewis et al. (2020) *"Retrieval-Augmented Generation for NLP"*
- **DuckDB:** Raasveldt & Mühleisen (2019) *"DuckDB Vectorized Execution Engine"*

The 5-Whys: Concluding Why We Need MinHash & LSH

Classroom interactive discussion based on Sakichi Toyoda's root-cause discovery framework.

HISTORICAL CONTEXT

Invented by **Sakichi Toyoda** (founder of Toyota Industries) for the Toyota Production System to dig past symptoms to root causes.

"Having no problems is the biggest problem of all. Ask 'why' five times about every matter." — Sakichi Toyoda

GUIDED DISCOVERY

Prove mathematically why SQL equality, wildcards, and word splitters fail, leaving **MinHash & LSH** as the ONLY scalable solution!

CLASSROOM CONCLUSION

We MUST use Entity Resolution, MinHash Signatures, and $O(N)$ LSH Banding!

Advance to Deck 02 for the math & live demo!

WHY #1 **Why are Netflix analytics corrupted?**

lew_hamilton_44 vs L3wis_Ham1lton register as 3 users, inflating metrics by 300%.

DECK 02 · TEXT MINING & SCALABLE DEDUPLICATION

Entity Resolution, MinHash Signatures & LSH Mathematics

Solving Question 1: Deduplicating 5 million noisy subscriber handles. From character bigrams to Broder's MinHash Theorem and $O(N)$ LSH Banding.

Bigrams

Jaccard Metric

Broder Theorem (1997)

LSH S-Curve

Demo 1

Dialogue 1: The 25-Trillion Pairwise Comparison Nightmare

A conversation on the engineering floor before building the text mining pipeline.

MIKE ROSS

"I'll just write a SQL loop with LIKE '%hamilton%'"

"Rachel, we have 5 million subscriber profiles. I can write a Python script with nested SQL LIKE statements to compare every handle against every other handle. It'll take maybe an hour, right?"

RACHEL GREEN

"Mike, your script will take 144 DAYS and burn our servers!"

*"Do the math, Mike! 5 million squared is **25 TRILLION matrix checks** (or **12.5 TRILLION unique pairs** via $N(N-1)/2$). Plus, LIKE '%hamilton%' completely misses L3wis_Ham1lton because of leet-speak. We need **Character Bigrams** for feature sets, **MinHash** to compress them, and **LSH Banding** to find candidates in linear $O(N)$ time!"*

INFOGRAPHIC: $O(N^2)$ PAIRWISE WALL VS $O(N)$ LSH BANDING

Naive SQL Pairwise Loop ($O(N^2)$)

$N^2 = 25$ Trillion checks (12.5T unique pairs) → **144 Days CPU Time**



MinHash + LSH Bucketing ($O(N)$)

5,000,000 items hashed into LSH buckets → **1.8 Seconds SIMD Time**

Character Bigram Set Fingerprinting

Converting unstructured text handles into mathematical sets.

- **Definition:** A *k-gram* (or bigram for $k=2$) is a contiguous sequence of k characters extracted from a normalized string.
- **Normalization:** Convert to lowercase, strip punctuation: `lew_hamilton` → `lewhamilton`.
- **Extraction:** Slide a window of length 2 across the string.
- `hamilton` → {**ha, am, mi, il, lt, to, on**}
- `h4milton` → {**h4, 4m, mi, il, lt, to, on**} (shares 5 bigrams despite leet-speak!).

WHY BIGRAMS BEAT WORD TOKENS

Word tokens fail on handles like `lewishamilton44` because there are no spaces. Character bigrams break down text into atomic sub-character fingerprints, making them immune to missing spaces, leet-speak, and typos.

Worked Example 1: Jaccard Set Similarity

Step-by-step master problem setup for computing Jaccard set similarity.

$$J(A, B) = |A \cap B| / |A \cup B|$$

PROBLEM STATEMENT

Evaluate Jaccard Similarity between two subscriber strings:

String A: "netflix"

String B: "netflx"

1. Extract 2-character bigram sets for both strings.
2. Calculate size of intersection $|A \cap B|$ and union $|A \cup B|$.
3. Determine if $J(A,B) \geq 0.55$ threshold cutoff for entity merging.

Worked Example 1 Solution: Jaccard Math

Full step-by-step mathematical breakdown and decision reveal.

STEP-BY-STEP SOLUTION

String A ("netflix"): Set A = {ne, et, tf, fl, li, ix} $\Rightarrow$ size 6

String B ("netflx"): Set B = {ne, et, tf, fl, lx} $\Rightarrow$ size 5

Intersection $A \cap B = \{ne, et, tf, fl\} \Rightarrow$ size 4

Union $A \cup B = \{ne, et, tf, fl, li, ix, lx\} \Rightarrow$ size 7

$J(\text{netflix}, \text{netflx}) = 4 / 7 = 0.571$ (57.1% match)

✓ Decision: $0.571 \geq 0.55$ threshold $\Rightarrow$ Flag as duplicate & merge into single canonical user ID!

Practice Problem 1: Jaccard Calculation

Solve this problem in your notes before advancing to the next slide for the solution reveal.

STUDENT CHALLENGE

Given handles **S1 = "hamilton"** and **S2 = "h4milton"**:

1. Extract 2-character bigram sets for both handles.
2. Find $|S1 \cap S2|$ and $|S1 \cup S2|$.
3. Compute $J(S1, S2)$. Is $J \geq 0.55$?

Practice Problem 1 Solution: Jaccard Math

Step-by-step mathematical solution and entity resolution decision.

SOLUTION REVEAL

$S1 \text{ bigrams} = \{ha, am, mi, il, lt, to, on\} \Rightarrow \text{size } 7$

$S2 \text{ bigrams} = \{h4, 4m, mi, il, lt, to, on\} \Rightarrow \text{size } 7$

$\text{Intersection } S1 \cap S2 = \{mi, il, lt, to, on\} \Rightarrow \text{size } 5$

$\text{Union } S1 \cup S2 = \{ha, am, h4, 4m, mi, il, lt, to, on\} \Rightarrow \text{size } 9$

$J(S1, S2) = 5 / 9 = 0.556 \geq 0.55 \text{ threshold}$

✓ Decision: MERGE ENTITIES! Leet-speak handle 'h4milton' is merged with 'hamilton'.

MinHash Signatures & Broder's Theorem

Compress huge bigram sets into tiny integer vectors while preserving Jaccard similarity.

- **Broder's Theorem (1997):** The probability that two sets A and B have the same minimum hash value under a random permutation is *exactly equal* to their Jaccard similarity:

$$P(h(A) = h(B)) = J(A, B)$$

- **Signature Vector:** Apply $K=100$ independent random hash functions $\{h_1, h_2, \dots, h_{100}\}$ to create a 100-integer signature for each user.

MINHASH PERMUTATION FORMULA

Hash Function: $h(x) = (a \cdot x + b) \bmod c$

- **x:** Row index of the shingle in the matrix.
- **a, b:** Co-prime random integers.
- **c:** A prime number greater than the total number of unique shingles (universe size **U**).

Note: The prime modulus $c > U$ ensures the hash function generates a proper, collision-free permutation of the row indices.

Reference Guide: [Arpit Bhavani MinHash Deep-Dive](#)

Worked Example 2: MinHash Permutations

Constructing MinHash signature rows using a collision-free prime modulus permutation.

PROBLEM STATEMENT

Given Universal Bigrams $U = \{0, 1, 2, 3\}$ (Universe size = 4)

Doc A = {0, 2}, Doc B = {2, 3}

Permutation Hash Function: $h_1(x) = (x + 3) \bmod 5$ (Prime $c = 5 > U$)

1. Compute $h_1(x)$ for all elements in Universal set U .
2. Compute MinHash signature values $\text{MinHash}_1(A)$ and $\text{MinHash}_1(B)$.
3. Check if $\text{MinHash}_1(A) == \text{MinHash}_1(B)$.

Worked Example 2 Solution: MinHash Signatures

Step-by-step evaluation of hash permutations across document sets.

STEP-BY-STEP SOLUTION

Hash Permutation $h_1(x) = (x + 3) \bmod 5$ ($c=5$ is prime, no collisions):

$x=0 \Rightarrow h_1(0) = 3$ | $x=1 \Rightarrow h_1(1) = 4$ | $x=2 \Rightarrow h_1(2) = 0$ | $x=3 \Rightarrow h_1(3) = 1$

The output mapping $\{3, 4, 0, 1\}$ is a bijection (proper permutation) of row indices.

MinHash_1(Doc A = $\{0, 2\}$): $\min(h_1(0), h_1(2)) = \min(3, 0) = 0$

MinHash_1(Doc B = $\{2, 3\}$): $\min(h_1(2), h_1(3)) = \min(0, 1) = 0$

✓ Match! $h_1(\text{Doc A}) = h_1(\text{Doc B}) = 0$. This agreement (both mapped to the shared shingle 2) estimates the true Jaccard overlap $J(A,B)$!

Practice Problem 2: MinHash Permutation

Solve this problem in your notes before checking the solution reveal on the next slide.

STUDENT CHALLENGE

Given Universal set $U = \{0, 1, 2, 3, 4\}$.

User A = $\{1, 3\}$, User B = $\{3, 4\}$.

Hash function: $h(x) = (3x + 1) \bmod 5$

1. Calculate $h(x)$ for all elements in set A and set B.
2. Find $\text{MinHash}(A)$ and $\text{MinHash}(B)$. Do they match?

Practice Problem 2 Solution: MinHash Math

Step-by-step min-hash calculation reveal.

SOLUTION REVEAL

User A elements: $x=1 \Rightarrow h(1)=4$ | $x=3 \Rightarrow h(3)=10 \bmod 5 = 0$

$\text{MinHash}(A) = \min(4, 0) = 0$

User B elements: $x=3 \Rightarrow h(3)=0$ | $x=4 \Rightarrow h(4)=13 \bmod 5 = 3$

$\text{MinHash}(B) = \min(0, 3) = 0$

✓ Match! $\text{MinHash}(A) = \text{MinHash}(B) = 0$ (Both contain element 3).

MinHash Generation: From Text to Signatures

Taking two real handles and tracking how they convert to shingles, characteristic columns, and signatures.

1. REAL SHINGLES & CHARACTERISTIC MATRIX

Doc A = "hamilton" $\Rightarrow$ Bigrams: {ha, am, mi, il, lt, to, on}

Doc B = "h4milton" $\Rightarrow$ Bigrams: {h4, 4m, mi, il, lt, to, on}

Universe U size = 9 distinct bigrams. We index them 0 to 8:

IDX	SHINGLE	DOC A	DOC B	H_1 $(X+3)\%11$	H_2 $(2X+1)\%11$	H_3 $(3X+5)\%11$
0	"ha"	1	0	3	1	5
1	"am"	1	0	4	3	8
2	"mi"	1	1	5	5	0
3	"il"	1	1	6	7	3
4	"lt"	1	1	7	9	6
5	"to"	1	1	8	0	9

2. MINHASH SIGNATURE MATRIX

To construct the signature, we find the minimum hash value for each row (shingle) present in the document that is not null (**minimum** hash value for each row).

SIGNATURE ROW	DOC A (MIN)
h_1 min value	3 ($x=0$)
h_2 min value	0 ($x=5$)
h_3 min value	0 ($x=2$)

SIGNATURE VECTORS (K=3)

Doc A Signature: **[3, 0, 0]^T**

Doc B Signature: **[0, 0, 0]^T**

Note how Jaccard similarity is 55.6%. In our small signature, 2/3 match (66.7%). $K=100$ converges to 55.6%.

Locality-Sensitive Hashing (LSH)

Group similar MinHash signatures into candidate buckets in $O(N)$ time.

- **Banding Technique:** Divide the 100-integer MinHash signature into $b=20$ bands of $r=5$ rows each ($b \times r = K = 100$).
- **Bucket Hashing:** Hash each band into a bucket. If two signatures match in *at least one band*, they become candidate pairs.
- **Candidate Probability Formula:**

$$P(\text{Candidate}) = 1 - (1 - s^r)^b$$

where $s = J(A,B)$ is the true Jaccard similarity.

LSH S-CURVE TUNING

By adjusting band count b and row count r , we tune the step-function threshold $s \approx (1/b)^{(1/r)}$ to filter out noise while guaranteeing near-100% recall for true duplicates.

Worked Example 3: LSH Banding Math

Evaluating candidate probability for high vs low Jaccard similarity.

PROBLEM STATEMENT

Given LSH configuration $b = 20$ bands, $r = 5$ rows ($K = 100$ signatures):

1. Calculate $P(\text{Candidate})$ for similar users with true Jaccard similarity $s = 0.80$.
2. Calculate $P(\text{Candidate})$ for dissimilar users with true Jaccard similarity $s = 0.20$.

Formula: $P(\text{Candidate}) = 1 - (1 - s^r)^b$

Worked Example 3 Solution: LSH S-Curve

Step-by-step candidate probability calculation.

STEP-BY-STEP SOLUTION

Case 1: Similar Users ($s = 0.80$)

$$s^r = 0.80^5 = 0.32768$$

$$P(\text{Candidate}) = 1 - (1 - 0.32768)^{20} = 1 - (0.67232)^{20} = \mathbf{0.9996 \text{ (99.96\% caught!)}}$$

Case 2: Dissimilar Users ($s = 0.20$)

$$s^r = 0.20^5 = 0.00032$$

$$P(\text{Candidate}) = 1 - (1 - 0.00032)^{20} = \mathbf{0.0064 \text{ (Filtered out!)}}$$

Practice Problem 3: LSH S-Curve Threshold

Calculate the candidate threshold and probability for different band configurations.

STUDENT CHALLENGE

An engineering team configures LSH with **b = 10 bands** and **r = 10 rows** ($K = 100$).

1. Calculate the approximate S-curve threshold $s \approx (1/b)^{(1/r)}$.
2. Calculate $P(\text{Candidate})$ for true similarity $s = 0.80$.

Practice Problem 3 Solution: LSH Math

Full mathematical calculation reveal.

SOLUTION REVEAL

1. Threshold $s \approx (1/10)^{(1/10)} = (0.1)^{0.1} = \mathbf{0.794}$ (stricter filter!)

2. For $s = 0.80$: $s^r = 0.80^{10} \approx 0.10737$

$$P(\text{Candidate}) = 1 - (1 - 0.10737)^{10} = 1 - (0.89263)^{10} \approx 1 - 0.316 = \mathbf{0.684}$$

(68.4% probability)

LSH Bucketing: Finding Candidate Pairs

Taking the signatures from Part 1 and running them through band bucketing and probability math.

1. HASHING SIGNATURE BANDS INTO BUCKETS

We use configuration: **b = 3 bands**, **r = 1 row per band** ($b \times r = K = 3$). Each band's sub-vector is passed to a hash function H_{band} :

BAND (ROW)	DOC A BLOCK	DOC B BLOCK	HASH FORMULA OUTPUT	RESULT
Band 1 (Row 1)	[3]	[0]	$H_1([3]) \rightarrow$ Bucket 3 $H_1([0]) \rightarrow$ Bucket 0	Mismatch
Band 2 (Row 2)	[0]	[0]	$H_2([0]) \rightarrow$ Bucket 12 $H_2([0]) \rightarrow$ Bucket 12	COLLISION!
Band 3 (Row 3)	[0]	[0]	$H_3([0]) \rightarrow$ Bucket 45	COLLISION!

2. THE PROBABILITY MATH

Let's calculate the exact probability of flagging Doc A and B as candidates under this LSH setup, given their true Jaccard similarity is **s = 0.556**:

1. Prob. that a single band matches:

$s^r = 0.556^1 = \mathbf{0.556}$ (55.6% match chance)

2. Prob. that a single band mismatches:

$1 - s^r = 1 - 0.556 = \mathbf{0.444}$

3. Prob. that they mismatch in ALL $b=3$ bands:

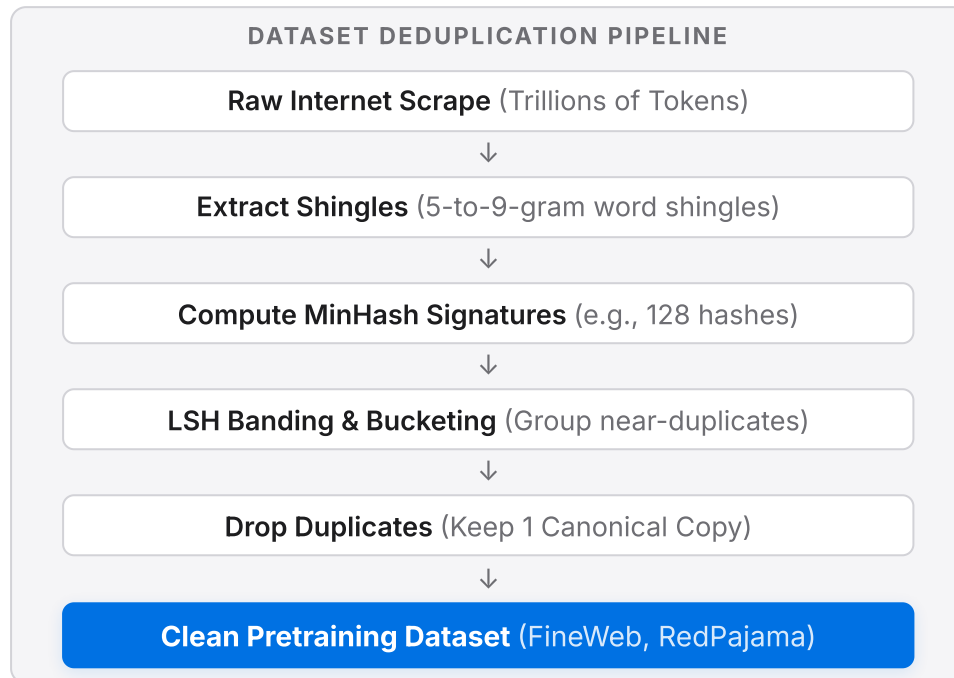
$(1 - s^r)^b = 0.444^3 = \mathbf{0.0875}$
(8.75% chance)

4. Prob. of matching in AT LEAST one band:

P(Candidate) = $1 - (1 - s^r)^b$
 $P = 1 - 0.0875 = \mathbf{0.9125}$ (91.25%)

Pretraining Datasets: How LSH Powers GenAI

MinHash & LSH are the industry standard for filtering trillions of raw tokens to train modern LLMs (like Llama 3).



SHINGLING

Documents are broken into overlapping words or tokens (usually 5-to-9-gram word shingles).

MINHASH

Millions of characters are compressed into a tiny signature vector of integers.

LSH

Signatures are grouped into bands and thrown into buckets. If two web pages share a bucket, they are flagged as a near-duplicate pair.

PRUNING

The pipeline drops duplicate pages and keeps exactly one clean, canonical version of the text.

DECK 03 · GRAPH ANALYTICS & NETWORK RANKING

Social Graph Mining, PageRank Iteration & Modularity Cliques

Solving Question 2: Finding the #1 top influencer in the network to feature on the live broadcast. Matrix power iteration and Louvain community detection.

Stanford Paper (1999)

Stochastic Matrices

Power Iteration

Spider Traps

Demo 2

Dialogue 2: The 500,000 Spam Bot Trap

A debate on executive desk before selecting the network influencer for live broadcast.

HARVEY SPECTER

"Just pick the user with the most followers!"

"Look at the database counter: @spam_bot_account has 500,000 followers! @Lew_hamilton only has 50,000. Put @spam_bot_account on the broadcast right now!"

MIKE ROSS

"Harvey, SQL COUNT is tricked by fake bot farms!"

*"Harvey, no! Those 500,000 followers are fake bot scripts created in a Russian click farm! @Lew_hamilton is followed by Toto Wolff, Christian Horner, and official F1 drivers. A link from an important person is worth 10,000 bot links. We need **PageRank** to compute link quality!"*

INFOGRAPHIC: IN-DEGREE SQL COUNT VS PAGERANK CENTRALITY

Naive SQL In-Degree (Count Quantity)

500,000 bot links → Spam Bot wins (Fake Influencer)

VS

PageRank Vector (Evaluates Link Quality)

Followed by F1 drivers → Lewis Hamilton wins (True Influencer)

Transition: From Clean Entities to Network Rank

We resolved 5M noisy handles into clean fan IDs. Now we address Question 2.

CRITICAL ENGINEERING QUESTION

"How do we find the #1 most important influencer in the fan network to feature on the live broadcast (without being fooled by fake bot followers)?"

WHY SQL IN-DEGREE FAILS

SQL `GROUP BY user_id COUNT(*)` measures link quantity, not link *quality*. A spam bot with 500k bot followers wins SQL COUNT, even though Lewis Hamilton (@lew_hamilton) has 50,000 real F1 driver and principal followers.

THE GRAPH REQUIREMENT

We need an algorithm where a link from an important node confers more rank than a link from an obscure node. We must model the network as a stochastic matrix and calculate its stationary rank distribution:

PageRank.

How PageRank Actually Works

The formula is just arithmetic. The idea is what matters. Build that first.

1

EVERY FAN GETS ONE VOTE

Every fan on Netflix distributes their vote equally across the people they follow. If Toto Wolff follows only Lewis Hamilton, Lewis gets 100% of Toto's vote. If Christian Horner follows Lewis and Max, each gets 50%.

2

A VOTE FROM AN IMPORTANT PERSON COUNTS MORE

If Toto Wolff himself has a high score, his vote for Lewis carries more weight than 10,000 votes from random bot accounts. Importance is **recursive** — your score depends on who votes for you.

3

KEEP REDISTRIBUTING UNTIL STABLE

We run the voting 50 times. After each round, scores shift. Eventually they stop changing — that is **convergence**. The final vector is the true ranking.

f

NOW THE FORMULA MAKES SENSE

$$v^{(k+1)} = d \cdot M \cdot v^{(k)} + (1-d)/N \cdot \mathbf{1}$$

TERM DECODER

d = 0.85

85% chance a fan follows a link. 15% chance they randomly jump to any profile.

M

The stochastic matrix.
Every column sums to 1.

(1-d)/N

Teleportation floor. Prevents any fan from reaching a score of zero.

$$\|v^{(k+1)} - v^{(k)}\| < 10^{-5}$$

Stop condition. Scores have stabilised.

SEMINAL PAPER

[Page et al. \(1999\)](#)

The PageRank Citation

Worked Example 4: PageRank Iteration 0 → 1

Four real fans in the network. We calculate Lewis Hamilton's rank after one round of voting.

THE NETWORK

4 fans. Each starts with equal score: $\mathbf{v}^{(0)} = [0.25, 0.25, 0.25, 0.25]^T$

Damping: $d = 0.85$ | Teleport floor: $(1-0.85)/4 = 0.0375$

Node 3 = Lewis Hamilton

Receives votes from:

Node 1 — Toto Wolff (follows only Lewis → $L(1) = 1$)

Node 2 — Christian Horner (follows Lewis + Max → $L(2) = 2$)

Node 4 — Max Verstappen (follows only Lewis → $L(4) = 1$)

Calculate Lewis's rank after Iteration 1: $\mathbf{v}_3^{(1)}$

WHY THIS SETUP?

Toto gives 100% of his vote to Lewis.

Christian gives 50% each to Lewis and Max.

Max gives 100% of his vote to Lewis.

The bot accounts vote for each other and ignore real users — they will score near zero.

Worked Example 4 Solution: PageRank Math

The number 0.56875 is not just arithmetic — it is Lewis Hamilton pulling ahead of every bot in the network.

STEP-BY-STEP CALCULATION

$$\text{Teleport floor} = (1 - 0.85) / 4 = \mathbf{0.0375}$$

$$v_3^{(1)} = 0.85 \cdot [(1/1) \cdot v_1^{(0)} + (1/2) \cdot v_2^{(0)} + (1/1) \cdot v_4^{(0)}] + 0.0375$$

$$v_3^{(1)} = 0.85 \cdot [0.25 + 0.125 + 0.25] + 0.0375$$

$$v_3^{(1)} = 0.85 \cdot (0.625) + 0.0375 = 0.53125 + 0.0375$$

$$v_3^{(1)} = \mathbf{0.56875}$$

WHAT 0.56875 MEANS

After just one round, Lewis Hamilton has already accumulated **56.9%** of all rank in the network.

The bot account, followed only by other bots, has redistributed rank among itself and stays near 0.25 — it cannot absorb quality votes.

AFTER CONVERGENCE (50 ITERATIONS)

Lewis reaches ~0.424 normalised score. The demo will show this live.

Practice Problem 4: PageRank Power Iteration

Solve this problem in your notes before checking the solution reveal on the next slide.

STUDENT CHALLENGE

3-node graph: Node A links to B. Node B links to C. Node C links to A and B.

Initial vector: $\mathbf{v}^{(0)} = [1/3, 1/3, 1/3]^T$ | Damping: $d = 0.85$.

1. What is out-degree $L(C)$?
2. Calculate Iteration 1 rank $\mathbf{v}_B^{(1)}$ for Node B.

Practice Problem 4 Solution: PageRank Math

Step-by-step power iteration solution reveal.

SOLUTION REVEAL

1. Node C links to A and B $\Rightarrow$ out-degree $L(C) = 2$.

2. Node B receives links from A (out-deg=1) and C (out-deg=2):

$$v_B^{(1)} = 0.85 \cdot [(1/1) \cdot v_A^{(0)} + (1/2) \cdot v_C^{(0)}] + (0.15 / 3)$$

$$v_B^{(1)} = 0.85 \cdot [1/3 + 1/6] + 0.05 = 0.85 \cdot (0.50) + 0.05$$

$$v_B^{(1)} = \mathbf{0.475}$$

Spider Traps & Dead Ends

Why the damping factor $d=0.85$ is mathematically necessary.

TRAP 1 — SPIDER TRAPS

If two nodes A and B only follow each other, all rank flowing into them gets trapped in a closed loop. Over many iterations, A and B absorb 100% of the network's rank, driving all other nodes to zero.

Mathematical Fix: $d=0.85$ ensures 15% of rank teleports out of the trap on every iteration, maintaining global stochastic equilibrium.

TRAP 2 — DEAD ENDS

A node with out-degree $L(i) = 0$ (no outgoing links) causes rank to leak out of the system. Column sum becomes 0, making matrix M non-stochastic.

Mathematical Fix: Replace dead-end columns with uniform probability vectors ($1/N$ for all rows).

Community Detection: Finding the Fan Tribes

PageRank tells you who the most important person is. Community detection tells you which tribe they belong to.

WHAT PAGERANK ANSWERS

"Who is the single most influential fan in the network?" → Lewis Hamilton. **One winner.**

WHAT COMMUNITY DETECTION ANSWERS

"Which fans naturally cluster together?" → The algorithm automatically groups 5 million fans into tribes: *F1 superfans, casual viewers, betting group, bot farms*. **No manual tagging required.**

THE LOUVAIN APPROACH (IN PLAIN ENGLISH)

Start with every fan in their own group. Greedily move fans into whichever neighbouring group makes the most internal connections. Repeat until no move improves the score. Runs in **$O(N \log N)$** — fast enough for 5 million nodes.

WHY THIS MATTERS HERE

Once PageRank identifies Lewis Hamilton as top influencer, community detection isolates the **bot farm cluster** — automatically quarantining 500,000 fake accounts from the genuine fan engagement metrics.

EXAM RELEVANCE

Understand the concept. You will not be asked to derive the modularity Q formula — that is graduate-level graph theory.

Live Demo 2 — Social Graph PageRank

Switch to Lab 2 on the sidebar now to demonstrate PageRank power iteration live.

INSTRUCTOR DEMO DIRECTIVE

1. Click "**Lab 2 — Social Graph Mining**" in the left sidebar.
2. Note that the live demo runs on the full **9-node follower network** (with Leclerc, Alonso, a spam bot, and VIP fans), whereas Worked Example 4 was a simplified 4-node subset for manual calculation.
3. Click "**Step Iteration**" 3 times to show score redistribution.
4. Click "**Converge**" to execute 50 iterations.
5. Show Lewis Hamilton achieving rank **~0.42** (converged top node circle size) compared to **0.0167** for the 500k spam bot!

DECK 04 · VECTOR EMBEDDINGS, RAG & LLM-DRIVEN DATA WAREHOUSING

Vector Spaces, Hugging Face LLM Course & LLM-Driven Data Mining

Solving Question 3: Deep dive into HF LLM Course Chapter 1 (Encoder/Decoder, Attention Q/K/V) and the 4 Core Frontiers of LLM-Driven Data Mining & Warehousing.

HF LLM Chapter 1

Text-to-SQL

Auto-ETL

AI Data Agents

DuckDB RAG

Dialogue 3: The Hallucinating AI Lawyer Trap

A tense crisis meeting after raw PDF contracts were fed directly into an LLM chatbot.

 **DONNA (TEAM LEAD)**

"The AI Chatbot gave us made-up names!"

"Legal uploaded 500GB of PDFs directly to an LLM chatbot and asked: 'Which paid subscribers discussed VIP passes before Day 100?'. It hallucinated a fake fan called 'Rahul Dravid Jr.' and got all the paid amounts wrong! Legal is screaming that AI is useless!"

 **LEAD DATA ARCHITECT (YOU)**

"LLMs predict words — they CANNOT do DB arithmetic!"

*"Donna, LLMs are trained to guess text — they aren't databases! They cannot perform exact SQL filters like `tier_paid > 500`. We need **pgvector** for semantic search, **PostgreSQL** for payment math, and **DuckDB** to JOIN them before the LLM sees a single verified fact!"*

FLOWCHART: DUAL-STORAGE GROUNDED RAG PIPELINE

PostgreSQL Ledger
(`tier_paid > $500`)

+

pgvector Store
(`cos sim ≥ 0.72`)

→

DuckDB SIMD JOIN
(`user_id match`)

→

Grounded LLM Prompt
(0 Hallucinations)

Text-to-SQL & Semantic Query Translation

How LLMs bridge natural language business intent directly to database execution.

NATURAL LANGUAGE QUESTION

"Show me top 5 VIP subscribers who paid more than \$500 in the first 100 days and discussed paddock passes."

LLM Processing: Schema Linking → Dialect Selection (PostgreSQL/DuckDB) → Constraint Mapping → CTE Generation.

GENERATED EXECUTABLE SQL

```
WITH vip_fans AS (  
  SELECT user_id, tier_paid_usd  
  FROM   subscriber_ledger  
  WHERE  tier_paid_usd > 500  
        AND days_active < 100  
)  
SELECT v.user_id, c.snippet, c.similarity  
FROM   vip_fans v  
JOIN   vector_search('paddock pass') c ON v.user_id = c.user_id  
ORDER BY v.tier_paid_usd DESC LIMIT 5;
```

Automated ETL & Schema Generation

Using LLMs to inspect dirty JSON streams and auto-generate dbt models & Star Schemas.

STEP 01 — STREAM INSPECTION

Raw JSON Schema Inference

LLM analyzes 40GB/day raw Twitch/Discord JSON streams to infer data types, high-cardinality keys, and nested structures.

STEP 02 — NORMALIZATION

Automated dbt SQL Generation

Generates modular dbt SQL transformations converting raw JSON into normalized 3NF staging tables.

STEP 03 — STAR SCHEMA

Fact & Dimension Modeling

Automatically creates `dim_subscribers`, `dim_influencers`, and `fact_streaming_events` tables with surrogate keys.

STEP 04 — QUALITY GUARDS

Automated Data Quality Tests

Injects automated dbt assertions: `not_null`, `unique`, and `foreign_key` checks to prevent dirty data loads.

Unstructured-to-Structured Knowledge Extraction

Deploying SLMs and VLMs to convert PDFs, images, and chat logs into clean SQL tables.

SLM EDGE EXTRACTION

Phi-3 / Gemma-2B

Sub-millisecond token extraction running on edge nodes. Extracts entities (usernames, transaction IDs) from Twitch chat streams at 10,000 msgs/sec.

VLM SCANNED PDF INGESTION

Florence-2 / GPT-4o Vision

Ingests scanned PDF contract agreements and handwritten receipts directly into PostgreSQL relational tables with OCR bounding-box precision.

STRUCTURED WAREHOUSE OUTPUT

Zero-Copy Relational Rows

Outputs validated JSON schema rows that land directly in DuckDB and PostgreSQL tables for immediate SIMD vectorized querying!

Autonomous AI Data Science Agents (Sharma et al. 2026)

Architecting autonomous data agents with ReAct planning, memory, and code sandboxes.

COMPONENT 1

Planning Module

Task decomposition using Chain-of-Thought (CoT), Tree-of-Thought (ToT), and ReAct (Reasoning + Acting) loops.

COMPONENT 2

Memory System

Working Memory (Attention context window) + Episodic Memory (Vector DB embeddings) + Semantic Memory (Knowledge Graphs).

COMPONENT 3

Tool Registry

Interfacing with Python interpreters, ML libraries (scikit-learn, XGBoost), and database engines (PostgreSQL, DuckDB).

COMPONENT 4

Execution Sandbox

Hugging Face LLM Course Chapter 1 Synthesis

Core taxonomy and concepts extracted from *Hugging Face LLM Course Chapter 1*.

MODEL TAXONOMY

Encoder vs Decoder vs Seq2Seq

BERT (Encoder) for classification & embeddings · GPT/Llama (Decoder) for text generation · T5 (Seq2Seq) for translation.

TOKENIZATION

BPE & Subword Shingling

Byte-Pair Encoding (BPE) merges frequent character pairs into vocabulary tokens. Maps strings to token ID vectors.

ATTENTION MATH

Query, Key, Value (Q, K, V)

$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$. Computes dynamic token context weights.

PRE-TRAINING LOSS

Causal Language Modeling (CLM)

$\mathcal{L}_{\text{CLM}} = - \sum \log P(x_i \mid x_{<i})$. Optimizes token probability over massive corpora.

Self-Attention Mechanism: Q, K, V Matrices

The mathematical engine inside transformer encoders and decoders (HF Chapter 1).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- **Query Matrix (Q):** What the current token is looking for.
- **Key Matrix (K):** What information other tokens hold.
- **Value Matrix (V):** The actual representation content passed forward.
- **Scale Factor $\sqrt{d_k}$:** Prevents dot products from growing excessively large in high dimensions (which would cause vanishing gradients in softmax).

🧠 ELI5: THE TOY BOX ANALOGY

You hold a red LEGO wheel (**Query Q**). You ask the toy box: *"Who matches me?"*

All toys show their tag (**Key K**): "I'm a red wheel!", "I'm a blue block!".

You pick the matching red wheel (**Softmax $Q \cdot K^T$**) and grab the piece (**Value V**) to build your race car!

What on Earth is Dual-Storage RAG Federation?

Demystifying hybrid RAG before looking at the query code.

PLAIN ENGLISH ANALOGY

"An LLM alone is like a smooth-talking lawyer who never checked the bank statements. PostgreSQL is the meticulous accountant who knows every penny. DuckDB is the CEO who forces the lawyer and accountant into a room to JOIN their facts!"

POSTGRESQL

The Accountant. Filters `tier_paid_usd > 500 AND days_active < 100` with exact math.

PGVECTOR

The Lawyer. Searches 1536-D embedding space for chats matching 'VIP paddock access pass'.

DUCKDB

The CEO. Joins accountant + lawyer results on `user_id`. Hands LLM verified facts.

Text as Dense Embedding Vectors

Converting unstructured text chunks into high-dimensional geometric spaces.

- **Dense Embeddings:** An embedding model maps a text chunk into a fixed-size dense vector v in 1536-dimensional space.
- **Semantic Proximity:** Text chunks with similar meaning map to vectors located close to each other in vector space.
- "VIP paddock access pass" and "pit lane credential discussion" produce vectors with tiny angular distance.
- **pgvector HNSW Indexing:** pgvector stores embeddings using **HNSW** (Hierarchical Navigable Small World) multi-layer graphs for sub-millisecond nearest neighbor search.

ELI5: THE PLAYGROUND MAP

Vectors are just map coordinates on a giant playground! All sports toys sit in the sports corner. "VIP Pass" and "Paddock Access" land right next to each other!

Worked Example 5: Cosine Vector Similarity

Problem setup for computing 3D vector dot product and angular similarity.

$$\cos(\theta) = (A \cdot B) / (||A|| \ ||B||)$$

PROBLEM STATEMENT

Query Vector A = **[0.80, 0.20, 0.50]** ("VIP paddock pass")

Document Chunk Vector B = **[0.75, 0.30, 0.45]** ("pit lane credential")

1. Calculate dot product $A \cdot B$.
2. Calculate vector magnitudes $||A||$ and $||B||$.
3. Calculate Cosine similarity $\cos(\theta)$. Is it ≥ 0.72 threshold cutoff?

Worked Example 5 Solution: Vector Math

Step-by-step vector dot product and norm calculation reveal.

STEP-BY-STEP SOLUTION

Dot product $A \cdot B = (0.80)(0.75) + (0.20)(0.30) + (0.50)(0.45) = 0.60 + 0.06 + 0.225 = 0.885$

$$||A|| = \sqrt{0.64 + 0.04 + 0.25} = \sqrt{0.93} \approx 0.964$$

$$||B|| = \sqrt{0.5625 + 0.09 + 0.2025} = \sqrt{0.855} \approx 0.925$$

$$\cos(\theta) = 0.885 / (0.964 \cdot 0.925) = 0.885 / 0.8917 = 0.9925 \text{ (99.25\% match!)}$$

✓ Decision: $0.9925 \geq 0.72$ threshold $\Rightarrow$ Chunk is semantically relevant!

Practice Problem 5: Cosine Vector Search

Solve this problem in your notes before checking the solution reveal on the next slide.

STUDENT CHALLENGE

Query Vector A = $[1, 0, 1]$.

Document Chunk Vector B = $[0, 1, 1]$.

1. Compute dot product $A \cdot B$.
2. Compute vector norms $\|A\|$ and $\|B\|$.
3. Compute Cosine similarity $\cos(\theta)$. Does it pass similarity threshold 0.72?

Practice Problem 5 Solution: Vector Search Math

Full step-by-step vector search threshold evaluation.

SOLUTION REVEAL

1. $A \cdot B = (1)(0) + (0)(1) + (1)(1) = 1$

2. $||A|| = \sqrt{1+0+1} = \sqrt{2} \approx 1.414$ | $||B|| = \sqrt{0+1+1} = \sqrt{2} \approx 1.414$

3. $\cos(\theta) = 1 / (\sqrt{2} \cdot \sqrt{2}) = 1 / 2 = 0.500$

Result: $0.500 < 0.72$ threshold $\Rightarrow$ REJECT CHUNK (Not semantically relevant).

DuckDB Vectorized Query Federation

Executing in-memory SIMD joins across relational SQL and vector stores.

WHY DUCKDB FOR RAG FEDERATION?

Traditional databases use the Volcano Iterator Model (tuple-at-a-time function calls). DuckDB uses a **Vectorized Columnar Execution Engine** (Raasveldt et al. 2019) processing vectors of 2,048 values in CPU SIMD registers.

DuckDB queries live PostgreSQL tables via `postgres_scan()` and pgvector index results simultaneously, executing an in-memory hash JOIN on `user_id` at zero copy cost.

DUCKDB FEDERATED SQL QUERY

```
SELECT g.user_id, g.user_handle, g.tier_paid_usd,
       v.chat_snippet, v.similarity_score
FROM   postgres_scan('subscriber_ledger') g
JOIN   vector_search('VIP paddock pass') v
      ON g.user_id = v.user_id
WHERE  g.tier_paid_usd > 500
      AND g.days_active < 100
      AND v.similarity_score > 0.72
ORDER BY g.tier_paid_usd DESC;
```

LIVE DEMO 3 & 4 — HYBRID RAG & WEB SCRAPING

Switch to Lab 3 & Lab 4 on the sidebar now to demonstrate Grounded RAG and Web Scraping live.

 INSTRUCTOR DEMO DIRECTIVE

1. Click "Lab 3 — Hybrid RAG Query" to show DuckDB JOIN resolving exact factual answers.
2. Click "Lab 4 — Web Scraping & Ingestion" to scrape [Hugging Face LLM Chapter 1](#) live.
3. Inspect raw DOM tree vs extracted concepts (Encoder, Decoder, Attention Q/K/V).
4. Stage extracted data into MinIO Data Lake & Vector Store!

DECK 05 · SOLUTIONS, ARCHITECTURE & CLOUD TRANSLATION

The Unified Stack: Activity Solutions, Cloud Maps & Rubric

Complete instructor solutions for the classroom activity, end-to-end architecture diagram, multi-cloud mapping, and 10-point rubric.

[Solutions Reveal](#)[End-to-End Diagram](#)[AWS Azure GCP](#)[Grading Rubric](#)

The Future of LLM-Driven Data Mining

Where autonomous data science agents are heading over the next decade.

KEY TRENDS FROM RESEARCH SURVEY (SHARMA 2026)

- **Multi-Agent Systems:** Specialized agents collaborating (e.g. ETL Agent + SQL Agent + ML Tuning Agent).
- **Multi-Modal Mining:** Mining tabular data, text logs, video streams, and audio simultaneously.
- **Domain-Specific Fine-Tuning:** Specialized agents for finance, healthcare, and sports streaming engines.
- **Execution Sandboxing:** Real-time Python code generation with self-correcting error traceback inspection.

THE CORE RULE FOR ENGINEERS

"AI Agents will handle repetitive coding and hyperparameter tuning, but human architects must design the data pipelines, guardrails, and ground-truth verification systems."

Task 1a Solution: OLTP Ingestion Trap

Why dumping 40GB/day of raw JSON into PostgreSQL crashes connection pools.

ROOT CAUSE ANALYSIS

PostgreSQL OLTP is optimized for ACID row transactions — not high-cardinality streaming bulk JSON writes. Streaming 40GB/day directly to Postgres:

- Locks buffer pools & exhausts connection limit (100-200 connections).
- Triggers Write-Ahead Log (WAL) amplification (40GB JSON → 160GB WAL).
- Starves incoming dashboard read queries, causing 20-min timeouts.

THE CORRECT ELT ARCHITECTURAL PATTERN

Stream raw JSON directly to **MinIO Object Storage** (Data Lake Staging Zone). Object storage has zero schema enforcement and infinite write throughput. A scheduled batch job extracts bigrams, runs LSH deduplication, and loads clean rows to PostgreSQL.

Task 1b Solution: SQL LIKE Deduplication Trap

Why regular expression wildcards fail at scale.

SQL LIKE LIMITATIONS

1. **No Leet-Speak Handling:** `LIKE '%hamilton%'` misses `L3wis_Ham1lton`.
2. **No Word-Order Resilience:** Misses `Hamilton_Lewis_F1`.
3. **Computational Wall:** $5M \times 5M = 25$ trillion pairwise comparisons ($O(N^2)$). Takes 144 days on standard DB servers.

LSH ALGORITHMIC FIX

1. Character 2-grams convert handles to feature sets.
2. MinHash compresses sets to 100-int signatures ($P = J(A,B)$).
3. LSH banding ($b=20, r=5$) hashes similar signatures into candidate buckets in $O(N)$ time.
4. Jaccard scoring evaluated ONLY on candidate pairs.

Task 1c Solution: LLM PDF Trap & RAG Fix

Why standalone LLMs hallucinate numbers and dates.

LLM DEFECT LLMs predict word tokens based on statistical probabilities. They cannot execute exact mathematical operations (`tier_paid_usd > 500`) or chronological filters (`days_active < 100`) over unstructured text. They hallucinate plausible-sounding names like "Rahul Dravid Jr."

VECTOR LIMIT Vector DB cosine search finds semantically related text about VIP passes, but CANNOT evaluate numerical inequality conditions.

DUAL-STORAGE FIX PostgreSQL evaluates exact math constraints; pgvector evaluates cosine semantic similarity; DuckDB executes inner JOIN on `user_id` ; LLM receives verified fact table as context.

Unified System Architecture Diagram

All three pipelines integrated into Netflix's real-time sports engine.

-
- LAYER 1 — STAGING** **MinIO Object Storage (Data Lake):** Raw JSON scraper streams + chat exports. Zero schema enforcement, unlimited write bandwidth.
-
- LAYER 2 — DEDUPLICATION** **Entity Resolution Engine:** PySpark/Python reads MinIO → Bigrams → MinHash (K=100) → LSH (b=20, r=5) → Jaccard ≥ 0.55 → Clean subscriber dimension loaded to PostgreSQL.
-
- LAYER 3 — TOPOLOGY & VECTOR** **PostgreSQL + pgvector:** PostgreSQL stores clean subscriber dimension, payment ledger, and network adjacency graph (PageRank). pgvector stores 1536-D chat embeddings with HNSW index.
-
- LAYER 4 — FEDERATION** **DuckDB In-Memory Engine:** Executes SIMD vectorized JOIN across PostgreSQL ledger and pgvector semantic matches on `user_id`.
-
- LAYER 5 — INTELLIGENCE** **Grounded LLM Prompting:** Formats verified DuckDB fact packet into sub-second live dashboard. Zero hallucinations.

AWS, Azure & GCP Service Translation

Every local Docker tool in our stack maps directly to managed services in all 3 clouds.

LOCAL TOOL (DOCKER)	STACK ROLE	AWS	AZURE	GCP
MinIO	Object Storage / Lake	Amazon S3	Azure Blob Storage	Cloud Storage (GCS)
PostgreSQL	Relational OLTP Store	Amazon RDS / Aurora	Azure DB for PostgreSQL	Cloud SQL for PostgreSQL
pgvector	Dense Vector Store	Amazon OpenSearch / Aurora pgvector	Azure AI Search	Vertex AI Vector Search
DuckDB	Federated Analytics	Amazon Athena / Redshift	Azure Synapse Serverless	Google BigQuery
PySpark / dbt	ETL & ER Pipelines	AWS Glue / DataBrew	Azure Data Factory	Cloud Dataflow / Dataform
Embedding Model	Text → Vectors	Bedrock Titan Embed	Azure OpenAI Embeddings	Vertex AI text- embedding

Task 3 Solution: Executive Memo

The 2-sentence technical answer for leadership.

MODEL ANSWER — EXECUTIVE TECHNICAL MEMO

"Standalone LLMs cannot guarantee accuracy for subscriber payment and pass queries because language models predict tokens rather than execute database arithmetic. To guarantee zero hallucinations, we deploy a dual-storage DuckDB architecture: PostgreSQL filters exact payment thresholds, pgvector retrieves semantically matching chat evidence, and DuckDB federates both on user_id before the model sees a single verified fact packet."

10-Point Activity Grading Rubric

Criteria for evaluating student handout submissions.

TASK	POINTS	FULL CREDIT CRITERIA
1a — OLTP Trap	2 pts	Mentions connection pool exhaustion, WAL amplification, or buffer starvation. Not just "it's slow."
1b — LIKE Trap	1 pt	Identifies leet-speak, word-order permutations, or $O(N^2)$ complexity bound.
1c — Dual Trap	2 pts	Correctly identifies BOTH: (i) LLMs cannot execute math/date filters, AND (ii) Vector DBs cannot evaluate relational threshold conditions.
Task 2 — Architecture	3 pts	Routes raw data to object storage first (1pt) · Decouples relational and vector paths (1pt) · Maps all 3 cloud providers correctly (1pt).
Task 3 — Executive Memo	2 pts	Explains the LLM defect AND the specific dual-storage + DuckDB federation fix in 2 clear sentences.

Core Principles of Unstructured Data Systems

Five foundational rules to take into your data engineering career.

- 01 **Stage raw streams in Object Storage:** Never write raw high-velocity JSON directly to OLTP databases. Data Lakes accept infinite scale.

- 02 **Scale string matching with LSH:** Use character bigrams, MinHash signatures, and LSH banding to drop pairwise comparison complexity from $O(N^2)$ to $O(N)$.

- 03 **Rank networks topologically:** In-degree count is vulnerable to bot farms. Use PageRank power iteration ($d=0.85$) to evaluate link quality.

- 04 **Never treat LLMs as databases:** LLMs predict words. Relational engines evaluate math. Use DuckDB to JOIN PostgreSQL and Vector Stores before prompting.

- 05 **Architecture is portable:** Local Docker tools (MinIO, Postgres, DuckDB) map 1:1 to AWS, Azure, and GCP managed services.